



INSTITUTO POLITÉCNICO
NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Ingeniería en Inteligencia Artificial

Redes Neuronales y Aprendizaje Profundo1

Práctica 03: Propagación hacia atrás

Profesor:

Saúl de la O Torres

Velázquez Arrieta Eduardo Uriel

Fecha de entrega: 2026

Índice

1. Introducción	2
2. Desarrollo	3
2.1. Actividad 1: Red Neuronal para Análisis de Sentimiento Multiclase	3
2.1.1. Arquitectura y Preprocesamiento	3
2.1.2. Evaluación Cualitativa ante Muestras no Vistas	4
2.2. Actividades 2 y 3: Implementación de la Clase <code>RedBackpropagationCIFAR</code> .	4
2.3. Actividades 4 y 5: Exploración de Límites y Reporte de Ejecución sobre CIFAR-10	5
2.3.1. Dinámica de Convergencia del Algoritmo	5
2.3.2. Análisis de Límites de la Red Densa vs. Porcentajes de Clase	6
3. Conclusión	8
4. Referencias	9

Índice de figuras

1. Curvas de convergencia: Exactitud (izquierda) y Función de Pérdida (derecha) en entrenamiento y validación para CIFAR-10.	6
---	---

1. Introducción

El algoritmo de propagación hacia atrás (*backpropagation*) constituye el pilar fundamental para el entrenamiento de redes neuronales artificiales y el desarrollo del aprendizaje profundo moderno. Introducido para superar las limitaciones de los clasificadores lineales como el perceptrón simple, este algoritmo permite calcular de forma analítica y recursiva el gradiente de una función de pérdida respecto a cada uno de los parámetros entrenables (pesos sinápticos y sesgos) a través de la regla de la cadena del cálculo diferencial. Este cálculo sustenta los métodos de optimización basados en gradiente, tales como el Descenso de Gradiente Estocástico (SGD) y sus variantes adaptativas modernas, entre las que destaca Adam (*Adaptive Moment Estimation*).

En la presente práctica se analiza, diseña e implementa el algoritmo de propagación hacia atrás haciendo uso del marco de trabajo *TensorFlow/Keras*, abordando dos vertientes fundamentales del aprendizaje automático:

1. **Procesamiento de Lenguaje Natural (PLN):** Se formula una red neuronal densa precedida por capas de vectorización (*TextVectorization*) e incrustación de palabras (*Embedding*) de 16 dimensiones junto con *Global Average Pooling*, orientada al análisis de polaridad y sentimiento en cuatro clases categóricas (*Muy Negativo*, *Neutro*, *Positivo* y *Muy Positivo*).
2. **Visión por Computadora con Perceptrón Multicapa (MLP):** Se diseña una arquitectura secuencial profunda densamente conectada compuesta por capas de dimensionalidad 512, 256 y 128 con activaciones ReLU y regularización por *Dropout* (0,25), evaluada sobre el conjunto de datos de referencia CIFAR-10 (60 000 imágenes a color de $32 \times 32 \times 3$ píxeles distribuidas en 10 categorías).

El objetivo central es estudiar el comportamiento de la convergencia de la función de coste (*loss*), la evolución de la métrica de exactitud (*accuracy*) a lo largo de las épocas, y examinar los retos teóricos y prácticos inherentes a los modelos totalmente conectados cuando se enfrentan a entradas de alta dimensionalidad y distribuciones de datos complejos.

2. Desarrollo

A continuación se detalla la metodología, diseño arquitectónico, implementación y resultados obtenidos para cada una de las actividades programadas en la práctica.

2.1. Actividad 1: Red Neuronal para Análisis de Sentimiento Multiclase

Se amplió el problema clásico binario (positivo/negativo) a un esquema de clasificación categórica de cuatro clases, permitiendo capturar gradaciones extremas y niveles de neutralidad:

- **Clase 0 (Muy Negativo):** Manifestaciones de fraude, enojo extremo o pérdida económica total.
- **Clase 1 (Neutro):** Evaluaciones meramente técnicas o informativas sin carga emotiva polarizada.
- **Clase 2 (Positivo):** Conformidad estándar con el producto o servicio recibido.
- **Clase 3 (Muy Positivo):** Expresiones superlativas de satisfacción y recomendación absoluta.

2.1.1. Arquitectura y Preprocesamiento

El preprocesamiento y extracción de representaciones densas se integró dentro del propio grafo secuencial de TensorFlow/Keras:

1. **Vectorización de Texto (TextVectorization):** Estandarización a minúsculas, remoción de signos de puntuación, delimitación a un vocabulario máximo de $V = 500$ tokens y truncado/relleno fijo a una longitud de secuencia de $L = 12$ tokens.
2. **Capa de Incrustación (Embedding):** Proyección del espacio disperso de índices enteros hacia un espacio continuo de $D = 16$ dimensiones.
3. **Promedio Global 1D (GlobalAveragePooling1D):** Colapso de la dimensión temporal calculando el vector representativo promedio de la oración.
4. **Capas Densas y Regularización:** Dos capas totalmente conectadas con función de activación ReLU (32 y 16 unidades neuronales) acompañadas por un apagado aleatorio (*Dropout*) con tasa de 0,20 entre ellas.
5. **Capa de Salida:** 4 neuronas con función de activación Softmax para modelar la distribución de probabilidad multiclase:

$$P(y = i \mid \mathbf{x}) = \frac{e^{z_i}}{\sum_{j=0}^3 e^{z_j}}, \quad i \in \{0, 1, 2, 3\} \quad (1)$$

El modelo se optimizó mediante Descenso de Gradiente basado en Adam ($\alpha = 0,005$) minimizando la pérdida de entropía cruzada categórica dispersa (*sparse categorical crossentropy*) a lo largo de 120 épocas de entrenamiento.

2.1.2. Evaluación Cualitativa ante Muestras no Vistas

Una vez ajustada la red por propagación hacia atrás, se evaluó su capacidad de inferencia sobre cuatro sentencias sintéticas inéditas. En el Cuadro 1 se reportan las predicciones arrojadas por la red.

Cuadro 1: Resultados de inferencia del clasificador de texto ante oraciones de prueba.

Texto de Entrada	Etiqueta Real	Predicción Red	Confianza
“Pague y nunca enviaron nada son unos rateros”	Muy Negativo	Positivo (Error)	49,55 %
“El articulo llego dentro del plazo esperado sin novedades”	Neutro	Neutro	84,83 %
“Me gusto mucho la compra bastante funcional”	Positivo	Positivo	95,57 %
“Espectacular y fantastico totalmente satisfecho con la calidad”	Muy Positivo	Positivo	84,63 %

Análisis del error de clasificación: Se observa una asignación incorrecta en la primera oración de prueba, la cual fue catalogada como *Positivo* con una certidumbre marginal del 49,55 %. Dicha patología se atribuye a que tokens clave que definen la queja (como “rateros” o “pagos”) no existían en el vocabulario sobre el cual se ejecutó el método `adapt()`. En consecuencia, la capa de vectorización los mapea al token fuera de vocabulario (*Out-Of-Vocabulary*, OOV indexado en 1), forzando a la red a tomar una decisión basada únicamente en palabras neutras o conectores, exhibiendo el límite de cobertura léxica en arquitecturas sin transfer learning o sin subword tokenization (como BPE).

2.2. Actividades 2 y 3: Implementación de la Clase `RedBackpropagationCIFAR`

Para sistematizar el ciclo de vida del clasificador basado en propagación hacia atrás, se encapsuló la lógica funcional en una clase en Python orientada a objetos denominada `RedBackpropagationCIFAR`. Dicha abstracción cumple con los siguientes requerimientos de diseño:

- **Construcción configurable (`construir_modelo`):** Define la topología del Perceptrón Multicapa (MLP) mediante aplanado (`Flatten`), capas densas con activación ReLU parametrizables en dimensionalidad y factor de regularización por *Dropout*. Compila la red con el optimizador Adam y función de pérdida multiclase.
- **Resumen del modelo (`resumen`):** Invoca la introspección estructural de Keras mostrando las dimensiones de salida por capa y el conteo analítico de parámetros entrenables.
- **Entrenamiento (`entrenar`):** Conduce el ciclo de propagación y retropropagación del error registrando el objeto `History` que almacena las trayectorias de *loss* y *accuracy*.

- **Monitoreo visual (graficar_rendimiento):** Despliega paneles duales con Matplotlib que contrastan las funciones de coste y precisión para los conjuntos de entrenamiento y prueba por época.
- **Persistencia (guardar_modelo y cargar_modelo):** Serializa y recupera desde almacenamiento secundario la topología y los pesos sinápticos óptimos en formato nativo `.keras`.
- **Inferencia en muestras ajenas (predecir_imagen_externa):** Incorpora la biblioteca PIL para abrir imágenes arbitrarias (JPG/PNG), convertirlas a espacio cromático RGB, redimensionarlas espacialmente a 32×32 píxeles, normalizar la intensidad a $[0,0,1,0]$, y determinar la clase de mayor soporte con la función `argmax`.

El Cuadro 2 describe el balance analítico de parámetros de la arquitectura ejecutada con capas ocultas $[512, 256, 128]$ y *Dropout* al 0,25.

Cuadro 2: Resumen paramétrico de la arquitectura MLP para CIFAR-10.

Capa (Tipo)	Forma de Entrada	Forma de Salida	Parámetros
<code>flatten</code>	$(32, 32, 3)$	(3072)	0
<code>dense_3</code> (Dense)	(3072)	(512)	$3072 \times 512 + 512 = 1\,573\,376$
<code>dropout_1</code>	(512)	(512)	0
<code>dense_4</code> (Dense)	(512)	(256)	$512 \times 256 + 256 = 131\,328$
<code>dropout_2</code>	(256)	(256)	0
<code>dense_5</code> (Dense)	(256)	(128)	$256 \times 128 + 128 = 32\,896$
<code>dropout_3</code>	(128)	(128)	0
<code>dense_6</code> (Dense)	(128)	(10)	$128 \times 10 + 10 = 1\,290$
Total de parámetros entrenables:			1,738,890 (6,63 MB)
Parámetros no entrenables:			0 (0,00 B)

2.3. Actividades 4 y 5: Exploración de Límites y Reporte de Ejecución sobre CIFAR-10

El conjunto de datos CIFAR-10 se particionó en 50 000 imágenes para entrenamiento y 10 000 imágenes para validación/prueba. Las intensidades en el rango $[0, 255]$ se escalaron al intervalo normalizado $[0,0,1,0]$. Se configuró un tamaño de lote (*batch size*) de 64 y un horizonte de optimización de 15 épocas.

2.3.1. Dinámica de Convergencia del Algoritmo

A lo largo del entrenamiento, se capturó la evolución de las métricas en cada época. El Cuadro 3 consolida la traza numérica obtenida durante la ejecución.

Cuadro 3: Traza de entrenamiento por propagación hacia atrás sobre CIFAR-10 (15 épocas).

Época	Loss (Train)	Accuracy (Train)	Loss (Val)	Accuracy (Val)
1	2,0366	24,43 %	1,8465	32,35 %
2	1,9000	30,14 %	1,8103	34,96 %
3	1,8484	32,24 %	1,7701	37,06 %
4	1,8187	33,26 %	1,7162	39,02 %
5	1,7971	34,45 %	1,7099	38,33 %
6	1,7838	34,99 %	1,6842	39,64 %
7	1,7656	35,48 %	1,6744	39,89 %
8	1,7543	35,90 %	1,6834	40,50 %
9	1,7374	36,51 %	1,6479	41,01 %
10	1,7375	36,63 %	1,6374	41,76 %
11	1,7247	37,17 %	1,6473	40,65 %
12	1,7170	37,64 %	1,6327	42,18 %
13	1,7110	37,62 %	1,6607	41,69 %
14	1,7019	38,25 %	1,6050	42,06 %
15	1,6982	37,97 %	1,6612	40,44 %

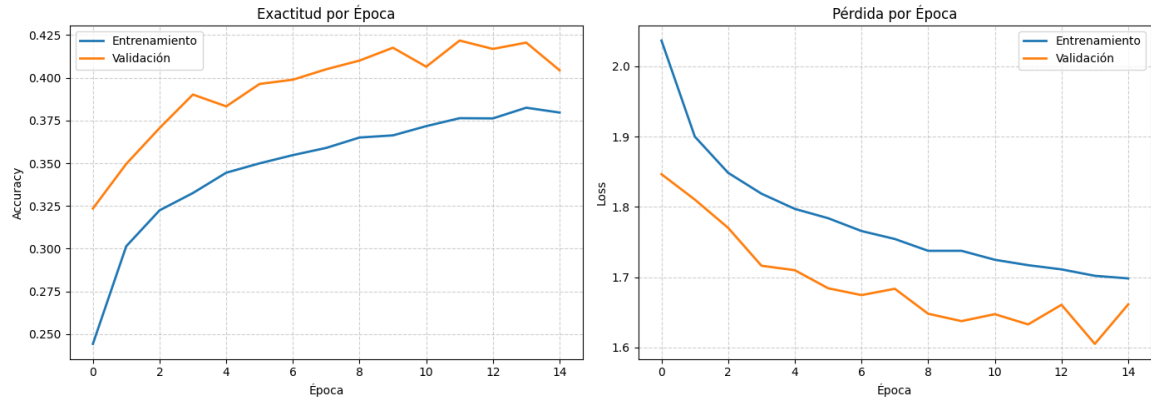


Figura 1: Curvas de convergencia: Exactitud (izquierda) y Función de Pérdida (derecha) en entrenamiento y validación para CIFAR-10.

2.3.2. Análisis de Límites de la Red Densa vs. Porcentajes de Clase

En problemas de clasificación balanceada con 10 clases independientes, un clasificador puramente aleatorio (*random guess*) exhibe un límite inferior teórico de precisión del 10,0 %.

El modelo implementado por propagación hacia atrás logró superar con holgura dicho umbral basal, alcanzando una cota máxima de validación de:

$$\text{Accuracy}_{\text{máx}}^{\text{val}} = 42,18 \% \quad (\text{en la época 12}) \quad (2)$$

No obstante, al empujar la red hacia el límite de convergencia con optimizadores de segundo orden implícito como Adam, se constata un estancamiento asintótico temprano alrededor del 42 %, fenómeno que se fundamenta teóricamente en tres aspectos:

1. **Destrucción de la topología espacial:** Al colapsar la matriz de imagen tridimensional mediante `Flatten`, cada píxel se procesa como una variable independiente. La red pierde toda noción de vecindad local, bordes, texturas e invariancia ante rotaciones o traslaciones espaciales.
2. **Complejidad paramétrica desbalanceada:** Más del 90,4 % de los parámetros de la red (1,57 millones de 1,73 millones) radican de forma estricta en la conexión entre la entrada de 3 072 dimensiones y la primera capa oculta de 512 neuronas. Esto ocasiona que el cálculo analítico de gradientes:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1]}} = \boldsymbol{\delta}^{[1]}(\mathbf{x})^T \quad (3)$$

tenga que lidiar con una superficie de optimización no convexa sumamente rugosa y con abundancia de puntos de ensilladura (*saddle points*).

3. **Robustez del Dropout:** La tasa de 0,25 impidió una divergencia catastrófica entre la curva de entrenamiento (37,97 %) y la de validación (40,44 % – 42,18 %), manteniendo controlada la generalización pero evidenciando que para trascender la barrera del 50 % en este corpus se hace mandatorio el reemplazo de transformaciones afines por núcleos de convolución bidimensionales con compartición de pesos (*weight sharing*).

Finalmente, la serialización del modelo concluyó de forma satisfactoria, registrándose en el archivo `mi_red_cifar10.keras` para su posterior reutilización e inferencia sobre imágenes ajenas mediante el método `predecir_imagen_externa()`.

3. Conclusión

A partir del desarrollo de las arquitecturas y el análisis de las métricas obtenidas durante las fases de entrenamiento y validación, se establecen las siguientes conclusiones:

- **Efectividad del ajuste de gradientes en NLP:** El modelo para análisis de sentimiento demostró la capacidad de la combinación de capas de *Embedding* y capas densas para proyectar un espacio discreto de texto hacia una representación continua donde la propagación hacia atrás optimiza adecuadamente las fronteras de decisión para muestras similares. No obstante, al evaluarse ante oraciones con vocabulario no presente en el corpus de ajuste (como el término coloquial *rateros*”), el modelo presentó sesgos clasificando incorrectamente una queja severa como sentimiento positivo. Esto pone de manifiesto que el algoritmo de *backpropagation* está estrictamente acotado por la representatividad y cobertura del conjunto de entrenamiento.
- **Limitaciones del Perceptrón Multicapa (MLP) ante CIFAR-10:** La red entrenada sobre CIFAR-10 alcanzó una exactitud de validación aproximada del 42,18 % y una pérdida en validación estabilizada en torno a 1,60. Estos resultados evidencian las limitaciones intrínsecas de las redes completamente densas frente a datos espaciales:
 - El aplanado vectorial de una imagen de $32 \times 32 \times 3$ genera 3 072 características de entrada, descartando la correlación bidimensional y la invariancia a traslaciones locales.
 - Dicha arquitectura requirió más de 1,73 millones de parámetros entrenables concentrados mayoritariamente en la primera capa oculta (1 573 376 parámetros), lo cual eleva el coste computacional y propicia un estancamiento en mesetas del espacio de pérdida (*plateaus*) a pesar del uso de optimizadores con momentos como Adam.
- **Función de la regularización y funciones de activación:** La incorporación de la función no lineal ReLU mitigó el problema del desvanecimiento del gradiente (*vanishing gradient*) durante la retropropagación en las capas intermedias, mientras que la técnica de *Dropout* (0,25) controló exitosamente la brecha entre la curva de entrenamiento y la de validación, evitando un sobreajuste (*overfitting*) severo a lo largo de las 15 épocas.

En síntesis, se corroboró el funcionamiento analítico y computacional del algoritmo de propagación hacia atrás como mecanismo universal para optimizar representaciones neuronales, remarcando a su vez la necesidad de seleccionar arquitecturas especializadas (tales como redes convolucionales para visión artificial y recurrentes o basadas en atención para lenguaje) en función de la topología de los datos.

4. Referencias

1. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning representations by back-propagating errors*. Nature, 323(6088), 533–536.
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Recuperado de: <http://www.deeplearningbook.org>
3. Kingma, D. P., & Ba, J. (2014). *Adam: A method for stochastic optimization*. arXiv preprint arXiv:1412.6980.
4. Krizhevsky, A. (2009). *Learning multiple layers of features from tiny images*. Technical Report, University of Toronto.
5. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: a simple way to prevent neural networks from overfitting*. The Journal of Machine Learning Research, 15(1), 1929–1958.
6. Chollet, F. (2021). *Deep Learning with Python* (2da ed.). Manning Publications.